DecodeLabs Data Analytics Internship - Project 3

SQL Relational Data Analysis & Extraction Report

Author: Monica Nyambura George
Batch: 2026
Domain: Data Analytics
Status: Completed & Validated

---

1. Project Objective

The goal of this project was to pivot from script-based dataframe analysis into relational database querying using structured SQL statements. By migrating the standardized transactional data into an in-memory SQLite database, we tested data constraints, executed filtered logical criteria, performed categorical multi-metric volumetric groupings, and provided clear, tabular business intelligence.

---

2. Core Relational Query Execution & Analysis

SQL Query 1: Global Metrics Audit

- **SQL Logic Applied:** Used foundational `COUNT(OrderID)` and `SUM(TotalPrice)` aggregation functions across the entire table.
- **Results Extracted:**
  - **Total Corporate Orders:** 1,200
  - **Gross Consolidated Revenue:** $1,264,761.96

SQL Query 2: Product Inventory & Order Volume Ranking

- **SQL Logic Applied:** Implemented a categorical `GROUP BY Product` function with a descending volume count constraint (`ORDER BY Order_Count DESC`).
- **Results Extracted:**

Product Line	Transaction Volume (Order Count)	Market Share Standing
Printer	181	Top Volume Driver
Tablet	179	High Demand Asset
Chair	178	High Demand Asset
Laptop	173	Mid-Tier Volume
Desk	170	Mid-Tier Volume
Monitor	163	Lower-Tier Volume
Phone	156	Baseline Floor Volume

- **Operational Note:** A crucial operational distinction exists here between transaction frequency and gross revenue. **Printers** lead the entire storefront in order velocity, being checked out in **181 separate transactions**. However, due to slightly higher quantities stacked per basket, **Chairs** brought in $195,620.11 in gross revenue, out-stepping Printers ($195,612.61) by just $7.50.

SQL Query 3: High-Value Transaction Isolation (> $2,000)

- **SQL Logic Applied:** Leveraged a strict `WHERE TotalPrice > 2000` filter row funnel to slice out lower-value spenders and isolate premium buyers.
- **Results Extracted:**
  - **Total VIP Premium Orders Found:** 180 out of 1,200 records (15% of total transactions).
  - **Top 3 Premium Invoices Isolated:**
    1. `ORD200789` (Customer `C57276` | Tablet) — **$3,456.40**
    2. `ORD201122` (Customer `C38840` | Monitor) — **$3,390.95**
    3. `ORD200632` (Customer `C67260` | Laptop) — **$3,390.80**

SQL Query 4: Settlement Channel Total Performance Metrics

- **SQL Logic Applied:** Structured a categorical grouping via `GROUP BY PaymentMethod` utilizing mathematical sum functions (`SUM(Quantity)` and `SUM(TotalPrice)`) to measure absolute volume scale.

- **Results Extracted:**

Payment Method Used	Total Units Sold	Total Revenue Generated	Market Volume Share
Credit Card	712	$263,847.63	Top Revenue Driver
Online	731	$262,442.94	High Volume Contributor
Cash	753	$259,786.29	Top Inventory Mover
Gift Card	675	$246,323.92	Mid-Tier Channel
Debit Card	664	$232,361.18	Baseline Volume Floor

- **Operational Note:** An interesting operational trade-off occurs here: **Cash** payment channels physically move the most volume out of the warehouse (**753 units**), closely followed by **Online** transactions (**731 units**). However, **Credit Cards** remain the primary capital engine for the storefront, capturing the absolute highest gross cash flow (**$263,847.63**) despite moving a lower volume of individual products (712 units).

---

3. Data Pipeline Implementation

The pipeline is designed using python's built-in `sqlite3` database package to run securely in a temporary memory allocation window. To trigger database instantiation and rerun the validation scripts:

```
python project3_sql.py
```